Bazele Inteligenţei Artificiale

Prof. Dr. Călin-Adrian POPA

Cursul 4

21 Octombrie 2025

4 Dimensionality reduction
4.1 The curse of dimensionality

- many machine learning problems involve thousands or even millions of features for each training instance
- not only do all these features make training extremely slow, but they can also make it much harder to find a good solution
- this problem is often referred to as the *curse of dimensionality*
- fortunately, in real-world problems, it is often possible to reduce the number of features considerably, turning an intractable problem into a tractable one

4.1 The curse of dimensionality

- reducing dimensionality does cause some information loss (just like compressing an image to JPEG can degrade its quality), so even though it will speed up training, it may make our system perform slightly worse
- it also makes our pipelines a bit more complex and thus harder to maintain
- so, if training is too slow, we should first try to train our system with the original data, before considering using dimensionality reduction
- in some cases, reducing the dimensionality of the training data may filter out some noise and unnecessary details and thus result in higher performance, but in general it won't; it will just speed up training

4.1 The curse of dimensionality

- apart from speeding up training, dimensionality reduction is also extremely useful for *data visualization*
- reducing the number of dimensions down to two (or three) makes it possible to plot a condensed view of a high-dimensional training set on a graph and often gain some important insights by visually detecting patterns, such as *clusters*
- moreover, data visualization is essential to communicate our conclusions to people who are not data scientists – in particular, decision makers who will use our results

4.1 The curse of dimensionality

- before we dive into specific dimensionality reduction algorithms, let's take a look at the two main approaches to reducing dimensionality: *projection* and *manifold learning*
- in most real-world problems, training instances are not spread out uniformly across all dimensions
- many features are almost constant, while others are highly correlated
- as a result, all training instances lie within (or close to) a much lower-dimensional *subspace* of the high-dimensional space
- let's look at an example; in Figure 1 we can see a 3D dataset represented by circles

4.1 The curse of dimensionality

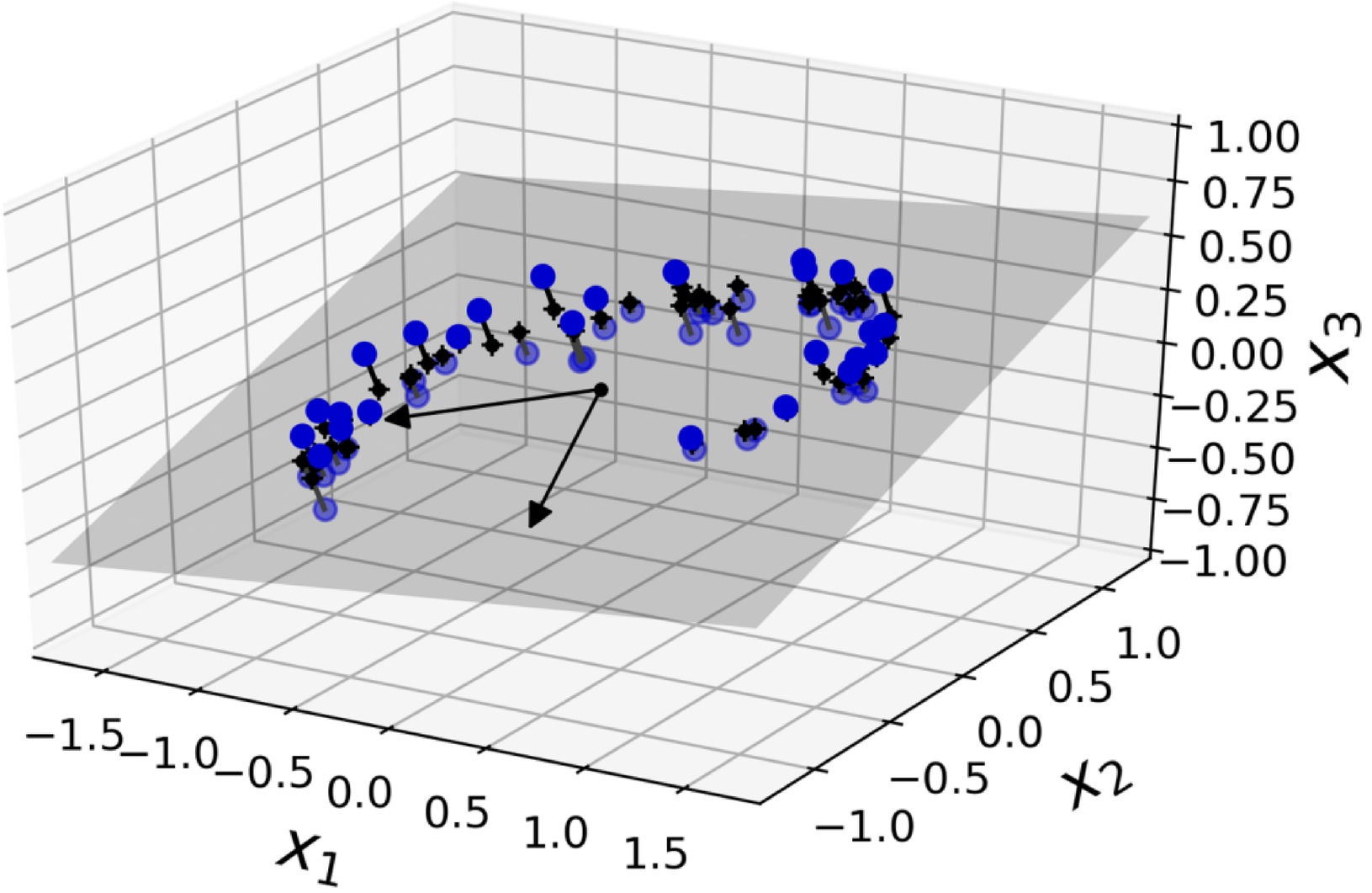


Figure 1: A 3D dataset lying close to a 2D subspace.

4.1 The curse of dimensionality

- notice that all training instances lie close to a plane: this is a lower-dimensional (2D) subspace of the high-dimensional (3D) space
- if we *project* every training instance perpendicularly onto this subspace (as represented by the short lines connecting the instances to the plane), we get the new 2D dataset shown in Figure 2
- we have just reduced the dataset's dimensionality from 3D to 2D
- note that the axes correspond to new features z_1 and z_2 (the coordinates of the projections on the plane)

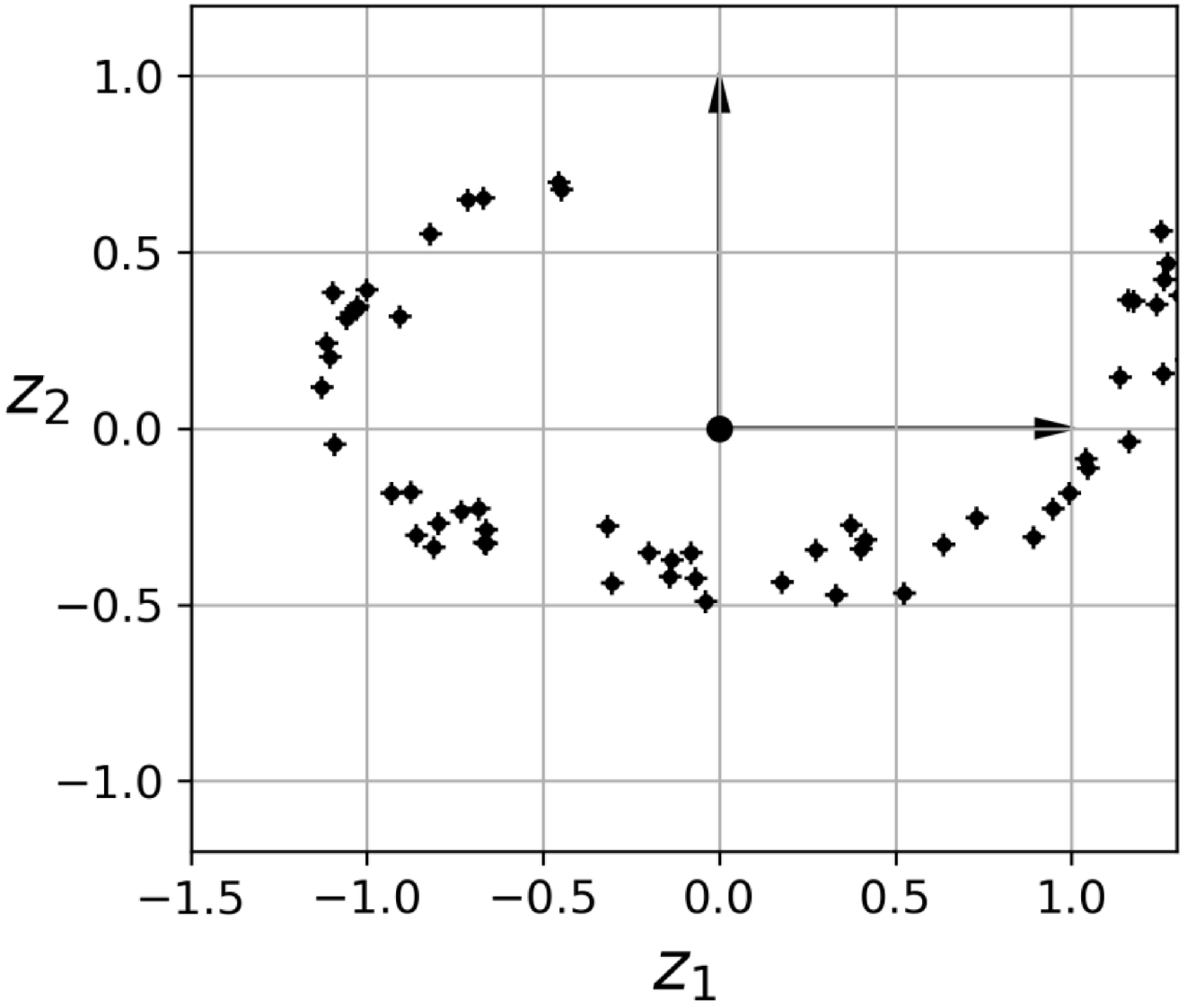


Figure 2: The new 2D dataset after projection.

4.1 The curse of dimensionality

- however, projection is not always the best approach to dimensionality reduction
- in many cases, the subspace may twist and turn, such as in the famous *Swiss roll* toy dataset, represented in Figure 3
- simply projecting onto a plane (e.g., by dropping x_3) would squash different layers of the Swiss roll together, as shown on the left side of Figure 4
- what we really want is to unroll the Swiss roll to obtain the 2D dataset on the right side of Figure 4

4.1 The curse of dimensionality

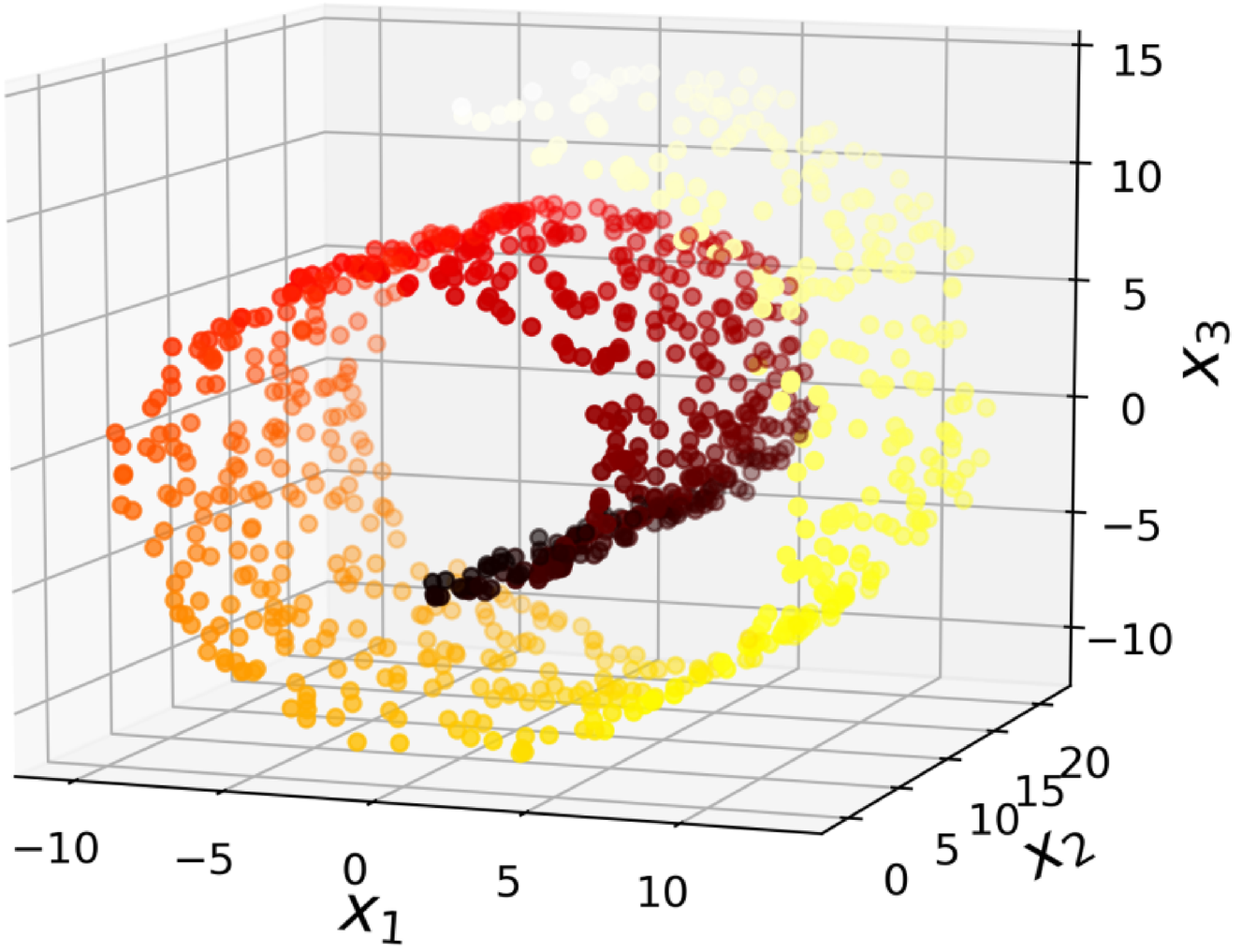


Figure 3: Swiss roll dataset.

4.1 The curse of dimensionality

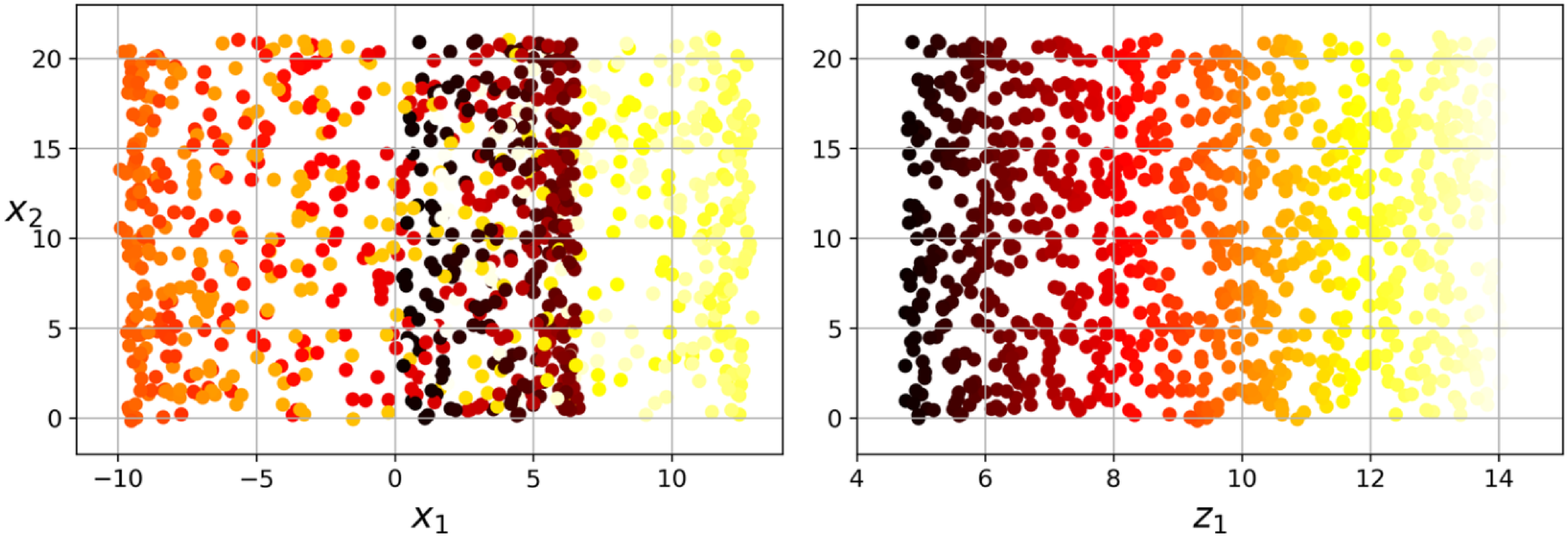


Figure 4: Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right).

4.1 The curse of dimensionality

- the Swiss roll is an example of a 2D *manifold*
- put simply, a 2D manifold is a 2D shape that can be bent and twisted in a higher-dimensional space
- more generally, an l-dimensional manifold is a part of a d-dimensional space (where $l < d$) that locally resembles an l-dimensional hyperplane
- in the case of the Swiss roll, $l = 2$ and $d = 3$: it locally resembles a 2D plane, but it is rolled in the third dimension

4.1 The curse of dimensionality

- many dimensionality reduction algorithms work by modeling the manifold on which the training instances lie; this is called *manifold learning*
- it relies on the *manifold assumption*, also called the *manifold hypothesis*, which states that most real-world high-dimensional datasets lie close to a much lower-dimensional manifold
- this assumption is very often empirically observed
- the manifold assumption is often accompanied by another implicit assumption: that the task at hand (e.g., classification or regression) will be simpler if expressed in the lower-dimensional space of the manifold
- for example, in the top row of Figure 5, the Swiss roll is split into two classes: in the 3D space (on the left), the decision boundary would be fairly complex, but in the 2D unrolled manifold space (on the right), the decision boundary is a straight line

4.1 The curse of dimensionality

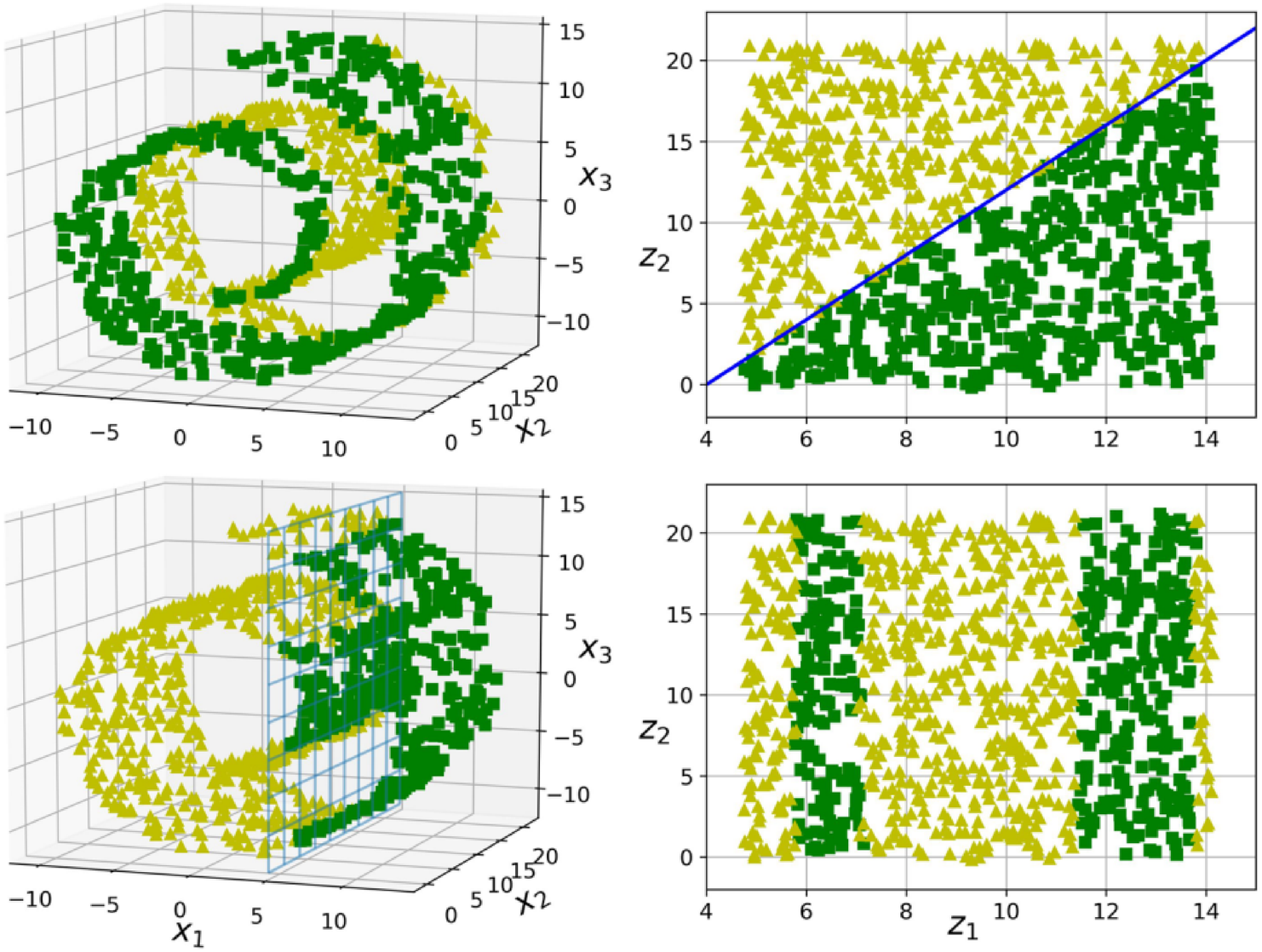


Figure 5: The decision boundary may not always be simpler with lower dimensions.

4.1 The curse of dimensionality

- however, this implicit assumption does not always hold
- for example, in the bottom row of Figure 5, the decision boundary is located at $x_1 = 5$
- this decision boundary looks very simple in the original 3D space (a vertical plane), but it looks more complex in the unrolled manifold (a collection of four independent line segments)
- in short, reducing the dimensionality of our training set before training a model will usually speed up training, but it may not always lead to a better or simpler solution; it all depends on the dataset
- we now have a good sense of what the curse of dimensionality is and how dimensionality reduction algorithms can fight it, especially when the manifold assumption holds
- the rest of this chapter will go through some of the most popular algorithms

4.2 Principal component analysis (PCA)

- *principal component analysis* (*PCA*) is by far the most popular dimensionality reduction algorithm
- first, it identifies the hyperplane that lies closest to the data, and then it projects the data onto it, just like in Figure 1
- before we can project the training set onto a lower-dimensional hyperplane, we first need to choose the right hyperplane
- for example, a simple 2D dataset is represented on the left in Figure 6, along with three different axes (i.e., 1D hyperplanes)
- on the right is the result of the projection of the dataset onto each of these axes
- as we can see, the projection onto the solid line preserves the *maximum variance*, while the projection onto the dotted line preserves very little variance and the projection onto the dashed line preserves an intermediate amount of variance

4.2 Principal component analysis (PCA)

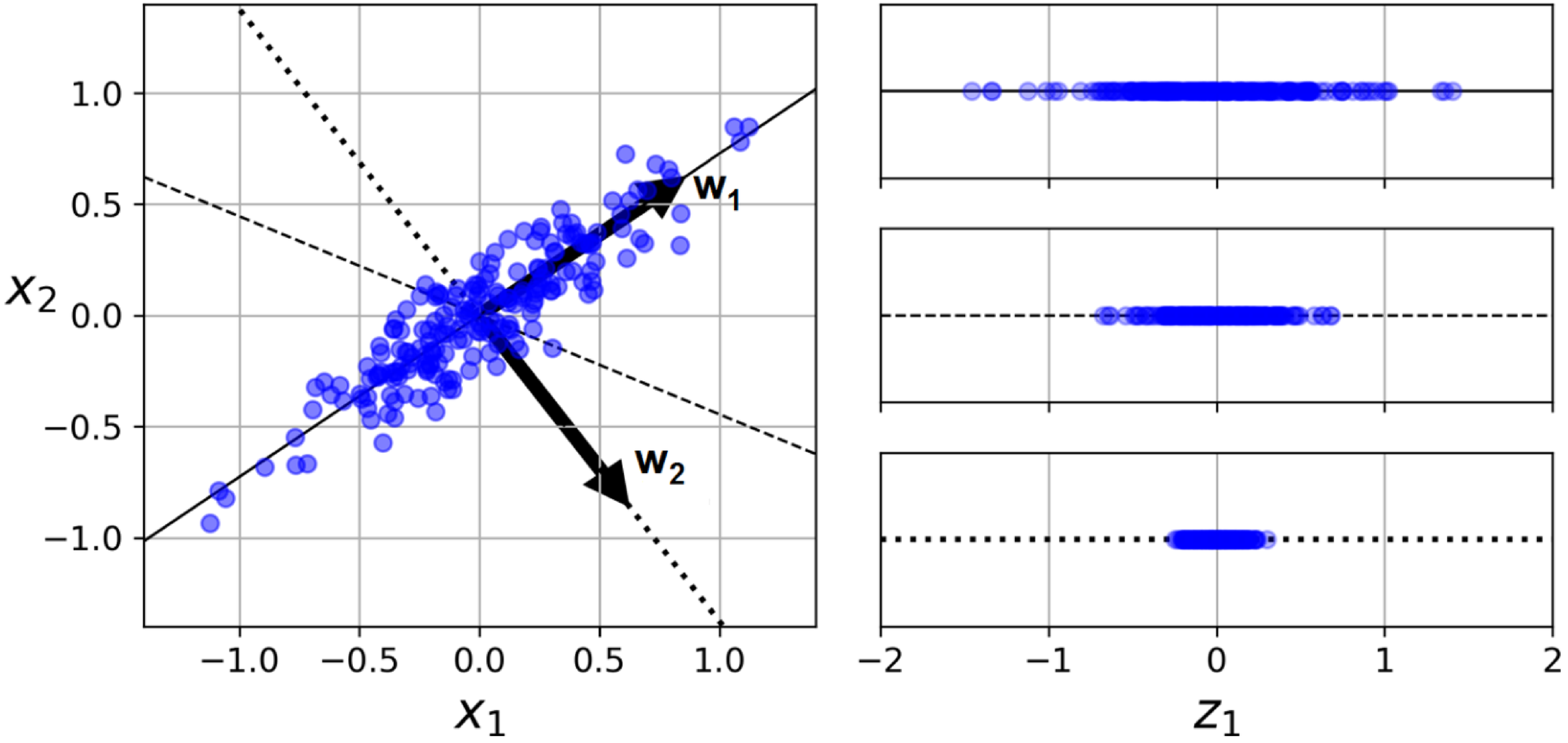


Figure 6: Selecting the subspace to project on.

4.2 Principal component analysis (PCA)

- it seems reasonable to select the axis that preserves the maximum amount of variance, as it will most likely lose less information than the other projections
- another way to justify this choice is that it is the axis that minimizes the mean squared distance between the original dataset and its projection onto that axis
- PCA identifies the axis that accounts for the largest amount of variance in the training set; in Figure 6, it is the solid line
- it also finds a second axis, *orthogonal* to the first one, that accounts for the largest amount of *remaining variance*
- in this 2D example, there is no choice: it is the dotted line
- if it were a higher-dimensional dataset, PCA would also find a third axis, orthogonal to both previous axes, and a fourth, a fifth, and so on – as many axes as the number of dimensions in the dataset

4.2 Principal component analysis (PCA)

- the *i*th axis is called the *i*th *principal component* (*PC*) of the data
- in Figure 6, the first PC is the axis on which vector $\boldsymbol{w}_1$ lies, and the second PC is the axis on which vector $\boldsymbol{w}_2$ lies
- in Figure 1, the first two PCs are the orthogonal axes on which the two arrows lie, on the plane, and the third PC is the axis orthogonal to that plane
- for each principal component, PCA finds a zero-centered unit vector pointing in the direction of the PC
- since two opposing unit vectors lie on the same axis, the direction of the unit vectors returned by PCA is not stable: if we perturb the training set slightly and run PCA again, the unit vectors may point in the opposite direction as the original vectors
- however, they will generally still lie on the same axes; in some cases, a pair of unit vectors may even rotate or swap (if the variances along these two axes are close), but the plane they define will generally remain the same

4.2 Principal component analysis (PCA)

- so how can we find the principal components of a training set?
- there are two commonly used definitions of PCA that give rise to the same algorithm
- PCA can be defined as the orthogonal projection of the data onto a lower-dimensional linear space, known as the *principal subspace*, such that the variance of the projected data is maximized
- equivalently, it can be defined as the linear projection that minimizes the average projection loss, defined as the mean squared distance between the data points and their projections

4.2 Principal component analysis (PCA)

- suppose we have an *unlabeled* dataset $\mathcal{D} = \{\boldsymbol{x}^{(i)} | i = 1, \ldots, n\}$, where $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$
- our goal is to project the data onto a space having dimensionality $l \leq d$, while *maximizing the variance* of the projected data
- to begin with, consider the projection onto a one-dimensional space ($l = 1$)
- we can define the direction of this space using a d-dimensional vector $\boldsymbol{w}_1$, which for convenience (and without loss of generality) we shall choose to be a unit vector so that $\boldsymbol{w}_1^\top \boldsymbol{w}_1 = 1$ (note that we are only interested in the direction defined by $\boldsymbol{w}_1$, not in the magnitude of $\boldsymbol{w}_1$ itself)
- each data point $\boldsymbol{x}^{(i)}$ is then projected onto a scalar value $\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)}$

4.2 Principal component analysis (PCA)

- the mean of the projected data is $\boldsymbol{w}_1^\top \overline{\boldsymbol{x}}$, where $\overline{\boldsymbol{x}}$ is the sample set mean given by:

$$\overline{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}^{(i)},$$

and the variance of the projected data is given by:

$$\begin{aligned}
\frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_1^\top \overline{\boldsymbol{x}})^2 &= \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_1^\top \overline{\boldsymbol{x}})(\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_1^\top \overline{\boldsymbol{x}})^\top \\
&= \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{w}_1^\top(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})^\top \boldsymbol{w}_1 \\
&= \boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1,
\end{aligned}$$

where $\boldsymbol{\Sigma}$ is the *data covariance matrix* defined by:

$$\boldsymbol{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})^\top.$$

4.2 Principal component analysis (PCA)

- we now have to maximize the projected variance $\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1$, with respect to $\boldsymbol{w}_1$
- we can trivially optimize this by letting $||\boldsymbol{w}_1||_2 \to \infty$, so this has to be a constrained maximization to prevent $||\boldsymbol{w}_1||_2 \to \infty$
- the appropriate constraint comes from the normalization condition $\boldsymbol{w}_1^\top \boldsymbol{w}_1 = 1$
- to enforce this constraint, we introduce a Lagrange multiplier that we shall denote by λ_1, and then make an unconstrained maximization of:

$$\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1 + \lambda_1 (1 - \boldsymbol{w}_1^\top \boldsymbol{w}_1).$$

- taking derivatives and equating to zero, we have:

$$\begin{aligned} 0 &= \frac{\partial}{\partial \boldsymbol{w}_1} \left(\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1 + \lambda_1 (1 - \boldsymbol{w}_1^\top \boldsymbol{w}_1) \right) \\ &= 2\boldsymbol{\Sigma} \boldsymbol{w}_1 - 2\lambda_1 \boldsymbol{w}_1. \end{aligned}$$

4.2 Principal component analysis (PCA)

- thus, we obtained:

$$\boldsymbol{\Sigma} \boldsymbol{w}_1 = \lambda_1 \boldsymbol{w}_1,$$

  which says that $\boldsymbol{w}_1$ must be an eigenvector of $\boldsymbol{\Sigma}$
- hence, the optimal direction onto which we should project the data is an eigenvector of the covariance matrix
- if we left-multiply by $\boldsymbol{w}_1^\top$ and use $\boldsymbol{w}_1^\top \boldsymbol{w}_1 = 1$, we see that the variance is given by:

$$\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1 = \lambda_1,$$

  and so the variance will be a maximum when we set $\boldsymbol{w}_1$ equal to the eigenvector having the *largest eigenvalue* λ_1
- this eigenvector is known as the *first principal component*

4.2 Principal component analysis (PCA)

- now, let us find another direction $\boldsymbol{w}_2$ to maximize the remaining variance
- as previously mentioned, this direction is chosen to be orthogonal to the first direction $\boldsymbol{w}_1$, i.e., $\boldsymbol{w}_2^\top \boldsymbol{w}_1 = 0$, and it will also be a unit vector, so $\boldsymbol{w}_2^\top \boldsymbol{w}_2 = 1$
- each data point $\boldsymbol{x}^{(i)}$ is now projected onto the following scalar value:

$$\boldsymbol{w}_2^\top (\boldsymbol{x}^{(i)} - \boldsymbol{w}_1 \boldsymbol{w}_1^\top \boldsymbol{x}^{(i)}) = \boldsymbol{w}_2^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_2^\top \boldsymbol{w}_1 \boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} = \boldsymbol{w}_2^\top \boldsymbol{x}^{(i)},$$

  where, for the last equality, we took into account the fact that $\boldsymbol{w}_2^\top \boldsymbol{w}_1 = 0$
- by the same argument, the mean of the projected data is $\boldsymbol{w}_2^\top (\overline{\boldsymbol{x}} - \boldsymbol{w}_1 \boldsymbol{w}_1^\top \overline{\boldsymbol{x}}) = \boldsymbol{w}_2^\top \overline{\boldsymbol{x}}$, and the variance of the projected data is given by:

$$\begin{aligned}
\frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}_2^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_2^\top \overline{\boldsymbol{x}})^2 &= \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}_2^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_2^\top \overline{\boldsymbol{x}})(\boldsymbol{w}_2^\top \boldsymbol{x}^{(i)} - \boldsymbol{w}_2^\top \overline{\boldsymbol{x}})^\top \\
&= \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{w}_2^\top (\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})^\top \boldsymbol{w}_2 \\
&= \boldsymbol{w}_2^\top \boldsymbol{\Sigma} \boldsymbol{w}_2.
\end{aligned}$$

4.2 Principal component analysis (PCA)

- now, we have to make an unconstrained maximization of:

$$\boldsymbol{w}_2^\top \boldsymbol{\Sigma} \boldsymbol{w}_2 + \lambda_2(1 - \boldsymbol{w}_2^\top \boldsymbol{w}_2) + \lambda_{12}(0 - \boldsymbol{w}_2^\top \boldsymbol{w}_1).$$

- taking derivatives and equating to zero, we have:

$$\begin{aligned} 0 &= \frac{\partial}{\partial \boldsymbol{w}_2}\left(\boldsymbol{w}_2^\top \boldsymbol{\Sigma} \boldsymbol{w}_2 + \lambda_2(1 - \boldsymbol{w}_2^\top \boldsymbol{w}_2) + \lambda_{12}(0 - \boldsymbol{w}_2^\top \boldsymbol{w}_1)\right) \\ &= 2\boldsymbol{\Sigma} \boldsymbol{w}_2 - 2\lambda_2 \boldsymbol{w}_2 - \lambda_{12} \boldsymbol{w}_1. \end{aligned}$$

- left-multiplying the last relation by $\boldsymbol{w}_1^\top$, we have:

$$2\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_2 - 2\lambda_2 \boldsymbol{w}_1^\top \boldsymbol{w}_2 - \lambda_{12} \boldsymbol{w}_1^\top \boldsymbol{w}_1 = 0.$$

4.2 Principal component analysis (PCA)

- we know that $\boldsymbol{\Sigma}\boldsymbol{w}_1 = \lambda_1 \boldsymbol{w}_1$, which implies that $\boldsymbol{w}_1^\top \boldsymbol{\Sigma} = \lambda_1 \boldsymbol{w}_1^\top$, because the covariance matrix $\boldsymbol{\Sigma}$ is symmetric: $\boldsymbol{\Sigma}^\top = \boldsymbol{\Sigma}$
- from $\boldsymbol{w}_1^\top \boldsymbol{w}_2 = 0$ and $\boldsymbol{w}_1^\top \boldsymbol{w}_1 = 1$, and the above relation, we obtain $\lambda_{12} = 0$
- this means that the solution is given by the eigenvector with the second largest eigenvalue λ_2, because the eigenvector with the largest eigenvalue λ_1 is $\boldsymbol{w}_1$:

$$\boldsymbol{\Sigma}\boldsymbol{w}_2 = \lambda_2 \boldsymbol{w}_2.$$

- the eigenvector $\boldsymbol{w}_2$ is the *second principal component*
- if we continue in this manner, by induction, we obtain that the optimal linear projection for which the variance of the projected data is maximized is defined by the l eigenvectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_l$ of the data covariance matrix $\boldsymbol{\Sigma}$ corresponding to the l largest eigenvalues $\lambda_1, \ldots, \lambda_l$

4.2 Principal component analysis (PCA)

- instead of maximizing the variance, we can define PCA as minimizing the *reconstruction error*
- in this setting, we have to find a linear and orthogonal projection of the high-dimensional data $\boldsymbol{x} \in \mathbb{R}^d$ to a low-dimensional subspace $\boldsymbol{z} \in \mathbb{R}^l$, such that the low-dimensional representation is a good approximation to the original data, in the following sense
- if we *project* or *encode* $\boldsymbol{x}$ to get $\boldsymbol{z} = \boldsymbol{W}^\top \boldsymbol{x}$, and then *unproject* or *decode* $\boldsymbol{z}$ to get $\hat{\boldsymbol{x}} = \boldsymbol{W}\boldsymbol{z}$, then we want $\hat{\boldsymbol{x}}$ to be as close as possible to $\boldsymbol{x}$ in ℓ_2 distance, where $\boldsymbol{W} \in \mathbb{R}^{d \times l}$ is the *projection matrix*

4.2 Principal component analysis (PCA)

- in other words, we would like to approximate each $\boldsymbol{x}^{(i)}$ by a low-dimensional representation, $\boldsymbol{z}^{(i)} \in \mathbb{R}^l$
- we assume that each $\boldsymbol{x}^{(i)}$ can be "explained" in terms of a weighted combination of basis vectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_l$, where each $\boldsymbol{w}_k \in \mathbb{R}^d$, $k = 1, \ldots, l$, and where the weights are given by $\boldsymbol{z}^{(i)} \in \mathbb{R}^l$, i.e., we assume that:

$$\boldsymbol{x}^{(i)} \approx \hat{\boldsymbol{x}}^{(i)},$$

  where:

$$\hat{\boldsymbol{x}}^{(i)} = \sum_{k=1}^{l} z_k^{(i)} \boldsymbol{w}_k = \boldsymbol{W}\boldsymbol{z}^{(i)}.$$

- the vector $\boldsymbol{z}^{(i)}$ is the low-dimensional representation of $\boldsymbol{x}^{(i)}$, and is known as the *latent vector*, since it consists of latent or hidden values that are not observed in the data

4.2 Principal component analysis (PCA)

- we can define the following *reconstruction error* or *projection error*:

$$\mathcal{L}(\boldsymbol{W}, \boldsymbol{Z}) = \frac{1}{n} \sum_{i=1}^{n} ||\boldsymbol{x}^{(i)} - \hat{\boldsymbol{x}}^{(i)}||_2^2 = \frac{1}{n} \sum_{i=1}^{n} ||\boldsymbol{x}^{(i)} - \boldsymbol{W}\boldsymbol{z}^{(i)}||_2^2,$$

  where the rows of $\boldsymbol{Z} \in \mathbb{R}^{n \times l}$ contain the low-dimension versions of the rows of $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, the *design matrix* of our dataset
- we want to minimize this error, subject to the constraint that $\boldsymbol{W}$ is an orthogonal matrix, because we want the corresponding basis vectors $\boldsymbol{w}_k \in \mathbb{R}^d$, $k = 1, \ldots, l$, to form an orthonormal basis in $\mathbb{R}^d$
- we also assume that $\overline{\boldsymbol{x}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}^{(i)} = \boldsymbol{0}$, which we can ensure by *centering* the data

4.2 Principal component analysis (PCA)

- let's again start by estimating $\boldsymbol{w}_1$; in this case, the reconstruction error is given by:

$$
\begin{aligned}
\mathcal{L}(\boldsymbol{w}_1, \boldsymbol{z}_1) &= \frac{1}{n}\sum_{i=1}^{n} ||\boldsymbol{x}^{(i)} - z_1^{(i)}\boldsymbol{w}_1||_2^2 \\
&= \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{x}^{(i)} - z_1^{(i)}\boldsymbol{w}_1)^\top (\boldsymbol{x}^{(i)} - z_1^{(i)}\boldsymbol{w}_1) \\
&= \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{x}^{(i)\top}\boldsymbol{x}^{(i)} - 2z_1^{(i)}\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} + (z_1^{(i)})^2 \boldsymbol{w}_1^\top \boldsymbol{w}_1) \\
&= \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{x}^{(i)\top}\boldsymbol{x}^{(i)} - 2z_1^{(i)}\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} + (z_1^{(i)})^2), \qquad (1)
\end{aligned}
$$

where, for the last equality, we used $\boldsymbol{w}_1^\top \boldsymbol{w}_1 = 1$, which is true by the orthonormality assumption

4.2 Principal component analysis (PCA)

- taking derivatives with respect to $z_1^{(i)}$ and equating to zero gives:

$$\begin{aligned} 0 &= \frac{\partial}{\partial z_1^{(i)}} \mathcal{L}(\boldsymbol{w}_1, \boldsymbol{z}_1) \\ &= \frac{1}{n}(-2\boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} + 2z_1^{(i)}), \end{aligned}$$

  thus $z_1^{(i)} = \boldsymbol{w}_1^\top \boldsymbol{x}^{(i)}$
- so the optimal embedding is obtained by orthogonally projecting the data onto $\boldsymbol{w}_1$
- plugging this back into (1) gives the loss for the weights:

$$\mathcal{L}(\boldsymbol{w}_1) = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{x}^{(i)\top}\boldsymbol{x}^{(i)} - (z_1^{(i)})^2) = \text{const.} - \frac{1}{n}\sum_{i=1}^{n}(z_1^{(i)})^2,$$

  where const. denotes terms that do not depend on $\boldsymbol{w}_1$

4.2 Principal component analysis (PCA)

- thus, we have to minimize:

$$-\frac{1}{n}\sum_{i=1}^{n}(z_1^{(i)})^2 = -\frac{1}{n}\sum_{i=1}^{n} z_1^{(i)} z_1^{(i)\top} = -\frac{1}{n}\sum_{i=1}^{n} \boldsymbol{w}_1^\top \boldsymbol{x}^{(i)} \boldsymbol{x}^{(i)\top} \boldsymbol{w}_1 = -\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1,$$

  where:

$$\boldsymbol{\Sigma} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})(\boldsymbol{x}^{(i)} - \overline{\boldsymbol{x}})^\top = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}^{(i)} \boldsymbol{x}^{(i)\top},$$

  because we assumed that $\overline{\boldsymbol{x}} = \boldsymbol{0}$

- minimizing $-\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1$ is the same as maximizing $\boldsymbol{w}_1^\top \boldsymbol{\Sigma} \boldsymbol{w}_1$, so we reached the same objective as in the first derivation, proving the equivalence of the two definitions

4.2 Principal component analysis (PCA)

- the question now is how to compute the eigenvectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_l$ of the data covariance matrix $\boldsymbol{\Sigma}$ corresponding to the l largest eigenvalues $\lambda_1, \ldots, \lambda_l$?
- maintaining the assumption that $\overline{\boldsymbol{x}} = \boldsymbol{0}$, the covariance matrix can be written as:

$$\boldsymbol{\Sigma} = \frac{1}{n} \boldsymbol{X}^\top \boldsymbol{X},$$

where $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ is the design matrix of our dataset

- recall that the *singular value decomposition* (*SVD*) of a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times d}$ is

$$\boldsymbol{A} = \boldsymbol{U} \boldsymbol{S} \boldsymbol{V}^\top,$$

where:

- the columns of $\boldsymbol{U} = [\boldsymbol{u}_1 | \ldots | \boldsymbol{u}_n] \in \mathbb{R}^{n \times n}$, the left singular vectors, are the orthonormal eigenvectors of $\boldsymbol{A}\boldsymbol{A}^\top \in \mathbb{R}^{n \times n}$
- the columns of $\boldsymbol{V} = [\boldsymbol{v}_1 | \ldots | \boldsymbol{v}_d] \in \mathbb{R}^{d \times d}$, the right singular vectors, are the orthonormal eigenvectors of $\boldsymbol{A}^\top \boldsymbol{A} \in \mathbb{R}^{d \times d}$
- the diagonal elements of $\boldsymbol{S} = \operatorname{diag}(s_1, \ldots, s_d)$, the singular values of $\boldsymbol{A}$, are such that $s_1^2 \geq \cdots \geq s_d^2$ are the eigenvalues of $\boldsymbol{A}^\top \boldsymbol{A}$

4.2 Principal component analysis (PCA)

- by taking $\boldsymbol{A} := \frac{1}{\sqrt{n}}\boldsymbol{X}$, we have that:
  - the first l columns of $\boldsymbol{V}$ from the SVD of $\boldsymbol{X}$ are the eigenvectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_l$ of $\boldsymbol{\Sigma} = \frac{1}{n}\boldsymbol{X}^\top \boldsymbol{X}$
  - the corresponding eigenvalues $\lambda_1 \geq \ldots \geq \lambda_l$ are $\frac{s_1^2}{n} \geq \cdots \geq \frac{s_l^2}{n}$, where $s_1, \ldots, s_l$ are the first l singular values of $\boldsymbol{X}$
- thus, $\boldsymbol{w}_k = \boldsymbol{v}_k$ and $\lambda_k = \frac{s_k^2}{n}$, $k = 1, \ldots, l$

4.3 Locally linear embedding (LLE)

- several *manifold learning* techniques have been proposed as *nonlinear dimensionality reduction* methods
- these algorithms implicitly assume that high-dimensional data lie on or near a low-dimensional nonlinear manifold embedded in the input space
- they aim to learn this manifold structure by finding a low-dimensional space that, in some way, preserves the local structure of the high-dimensional input data

4.3 Locally linear embedding (LLE)

- *locally linear embedding* (*LLE*) is a manifold learning technique that does not rely on projections, like the PCA algorithm does
- in a nutshell, LLE works by first measuring how each training instance linearly relates to its *closest neighbors*, and then looking for a low-dimensional representation of the training set where these local relationships are best preserved
- this approach makes it particularly good at unrolling twisted manifolds, especially when there is not too much noise

4.3 Locally linear embedding (LLE)

- for example, we can use locally linear embedding to unroll the Swiss roll
- the resulting 2D dataset is shown in Figure 7
- as we can see, the Swiss roll is completely unrolled, and the distances between instances are locally well preserved
- however, distances are not preserved on a larger scale: the left part of the unrolled Swiss roll is stretched, while the right part is squeezed
- nevertheless, LLE did a pretty good job at modeling the manifold

4.3 Locally linear embedding (LLE)

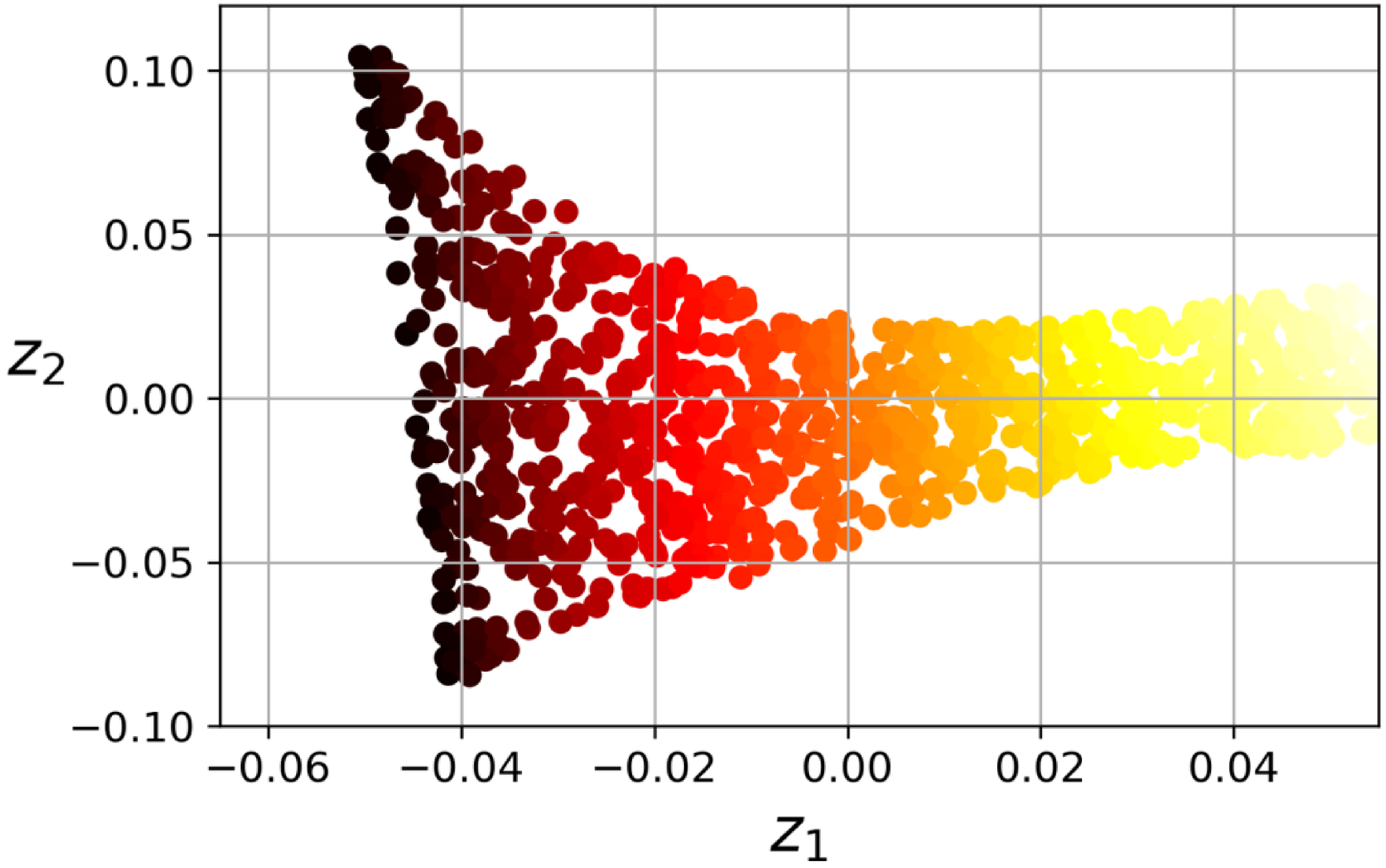


Figure 7: Unrolled Swiss roll using LLE.

4.3 Locally linear embedding (LLE)

- here's how LLE works: for each training instance $\boldsymbol{x}^{(i)}$, the algorithm identifies its k *closest neighbors* (in the preceding example, $k = 10$), then tries to reconstruct $\boldsymbol{x}^{(i)}$ as a linear function of these neighbors
- more specifically, it finds the weights $w_{i,j}$ such that the squared distance between $\boldsymbol{x}^{(i)}$ and $\sum_{j=1}^{n} w_{i,j}\boldsymbol{x}^{(j)}$ is as small as possible, assuming $w_{i,j} = 0$ if $\boldsymbol{x}^{(j)}$ is not one of the k closest neighbors of $\boldsymbol{x}^{(i)}$
- thus, the first step of LLE is the following constrained optimization problem:

$$\widehat{\boldsymbol{W}} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \sum_{i=1}^{n} \left(\boldsymbol{x}^{(i)} - \sum_{j=1}^{n} w_{i,j}\boldsymbol{x}^{(j)} \right)^2$$

$$\text{subject to} \begin{cases} w_{i,j} = 0, & \text{if } \boldsymbol{x}^{(j)} \text{ is not one of the } k \text{ closest neighbors of } \boldsymbol{x}^{(i)} \\ \sum_{j=1}^{n} w_{i,j} = 1, & \text{for } i = 1, 2, \ldots, n, \end{cases}$$

  where $\boldsymbol{W} \in \mathbb{R}^{n \times n}$ is the weight matrix containing all the weights $w_{i,j}$
- the second constraint simply normalizes the weights for each training instance $\boldsymbol{x}^{(i)}$

4.3 Locally linear embedding (LLE)

- after this step, the weight matrix $\widehat{\boldsymbol{W}}$ (containing the weights $\hat{w}_{i,j}$) encodes the local linear relationships between the training instances
- the second step is to map the training instances into an l-dimensional space (where $l < d$) while preserving these local relationships as much as possible
- if $\boldsymbol{z}^{(i)}$ is the image of $\boldsymbol{x}^{(i)}$ in this l-dimensional space, then we want the squared distance between $\boldsymbol{z}^{(i)}$ and $\sum_{j=1}^{n} \hat{w}_{i,j}\boldsymbol{z}^{(j)}$ to be as small as possible
- this idea leads to the following unconstrained optimization problem:

$$\widehat{\boldsymbol{Z}} = \underset{\boldsymbol{Z}}{\operatorname{argmin}} \sum_{i=1}^{n} \left(\boldsymbol{z}^{(i)} - \sum_{j=1}^{n} \hat{w}_{i,j}\boldsymbol{z}^{(j)} \right)^2 .$$

- it looks very similar to the first step, but instead of keeping the instances fixed and finding the optimal weights, we are doing the reverse: keeping the weights fixed and finding the optimal position of the instances' images in the low-dimensional space
- note that $\boldsymbol{Z} \in \mathbb{R}^{n \times l}$ is the matrix containing all $\boldsymbol{z}^{(i)}$

4.3 Locally linear embedding (LLE)

- the LLE algorithm has the following computational complexity: $\mathcal{O}(n \log(n) d \log(k))$ for finding the k closest neighbors, $\mathcal{O}(ndk^3)$ for optimizing the weights, and $\mathcal{O}(ln^2)$ for constructing the low-dimensional representations
- unfortunately, the n^2 in the last term makes this algorithm scale poorly to very large datasets

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- there is a class of algorithms for visualization that allow for much more complex mappings, and often provide better visualizations
- a particularly useful one is the *t-SNE* algorithm
- it reduces dimensionality while trying to keep similar instances close and dissimilar instances apart
- it is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- manifold learning algorithms are mainly aimed at visualization, and so are rarely used to generate more than two new features
- some of them, including t-SNE, compute a new representation of the training data, but don't allow transformations of new data
- this means these algorithms cannot be applied to a test set: rather, they can only transform the data they were trained for
- manifold learning can be useful for exploratory data analysis

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- the idea behind t-SNE is to find an l-dimensional representation of the data that preserves the distances between points as best as possible
- t-SNE starts with a random l-dimensional representation for each data point, and then tries to make points that are close in the original feature space closer, and points that are far apart in the original feature space farther apart
- t-SNE puts more emphasis on points that are close by, rather than preserving distances between far-apart points
- in other words, it tries to preserve the information indicating which points are neighbors to each other

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- t-SNE is a very popular nonconvex technique for learning low-dimensional embeddings
- it extends the earlier *stochastic neighbor embedding* (*SNE*) method, so we first describe SNE, before describing the t-SNE extension
- the basic idea in SNE is to convert high-dimensional Euclidean *distances* into *conditional probabilities* that represent *similarities*
- more precisely, we define $p_{j|i}$ to be the probability that point j would pick point i as its neighbor, if neighbors were picked in proportion to their probability under a Gaussian centered at $\boldsymbol{x}^{(i)}$:

$$p_{j|i} = \frac{\exp\left(-\frac{1}{2\sigma_i^2}||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||_2^2\right)}{\sum_{\substack{k=1 \\ k\neq i}}^{n} \exp\left(-\frac{1}{2\sigma_i^2}||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(k)}||_2^2\right)}.$$

- here, σ_i^2 is the variance for data point i, which can be used to "magnify" the scale of points in dense regions of input space, and diminish the scale in sparser regions; we discuss how to estimate the length scales σ_i^2 shortly

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- let $\boldsymbol{z}^{(i)}$ be the low-dimensional embedding representing $\boldsymbol{x}^{(i)}$
- we define similarities in the low-dimensional space in an analogous way:

$$q_{j|i} = \frac{\exp\left(-||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}||_2^2\right)}{\sum_{\substack{k=1 \\ k \neq i}}^{n} \exp\left(-||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(k)}||_2^2\right)}.$$

- in this case, the variance is fixed to a constant; changing it would just rescale the learned map, and not change its topology

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- if the embedding is a good one, then $q_{j|i}$ should match $p_{j|i}$
- therefore, SNE defines the objective to be:

$$\mathcal{L} = \sum_{i=1}^{n} KL(P_i||Q_i) = \sum_{i=1}^{n}\sum_{j=1}^{n} p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}, \tag{2}$$

where:

- P_i is the conditional distribution over all other data points given $\boldsymbol{x}^{(i)}$
- Q_i is the conditional distribution over all other latent points given $\boldsymbol{z}^{(i)}$
- $KL(P_i||Q_i)$ is the Kullback-Leibler divergence between the distributions

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- the *Kullback-Leibler divergence*, also known as the *information gain* or *relative entropy*, between two distributions p and q is a *divergence measure* which quantifies how far is q from p
- for discrete distributions, the Kullback-Leibler divergence is defined as follows:

$$KL(p||q) = \sum_{k=1}^{n} p_k \log \frac{p_k}{q_k}.$$

- we can rewrite the Kullback-Leibler divergence as follows:

$$KL(p||q) = \underbrace{\sum_{k=1}^{n} p_k \log p_k}_{-H(p)} - \underbrace{\sum_{k=1}^{n} p_k \log q_k}_{-H(p,q)}.$$

- we recognize the first term as the negative entropy, and the second term as the cross entropy
- thus, we can interpret the Kullback-Leibler divergence as the “extra number of bits” we need to pay when compressing data samples from p using the incorrect distribution q as the basis of our coding scheme

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- note that (2) is an asymmetric objective; in particular, there is a large cost if a small $q_{j|i}$ is used to model a large $p_{j|i}$
- this objective will prefer to pull distant points together, rather than push nearby points apart
- we can get a better idea of the geometry by looking at the gradient for each embedding vector, which is given by:

$$\nabla_{\boldsymbol{z}^{(i)}} \mathcal{L}(\boldsymbol{Z}) = 2 \sum_{j=1}^{n} (\boldsymbol{z}^{(j)} - \boldsymbol{z}^{(i)})(p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j}).$$

- thus, points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's
- although this is an intuitive objective, it is not convex; nevertheless, it can be minimized using SGD
- in practice, it helps to add Gaussian noise to the embedding points, and to gradually anneal the amount of noise

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- there is a slightly simpler version of SNE that minimizes a single Kullback-Leibler divergence between the joint distribution P in high-dimensional space and Q in low-dimensional space:

$$\mathcal{L} = KL(P||Q) = \sum_{\substack{i,j=1 \\ i<j}}^{n} p_{ij} \log \frac{p_{ij}}{q_{ij}}. \tag{3}$$

- this is called *symmetric SNE*
- the obvious way to define p_{ij} is to use:

$$p_{ij} = \frac{\exp\left(-\frac{1}{2\sigma^2}||\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}||_2^2\right)}{\sum_{\substack{k,k'=1 \\ k<k'}}^{n} \exp\left(-\frac{1}{2\sigma^2}||\boldsymbol{x}^{(k)} - \boldsymbol{x}^{(k')}||_2^2\right)}.$$

- we can define q_{ij} similarily

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- the corresponding gradient becomes:

$$\nabla_{\boldsymbol{z}^{(i)}} \mathcal{L}(\boldsymbol{Z}) = 2 \sum_{j=1}^{n} (\boldsymbol{z}^{(j)} - \boldsymbol{z}^{(i)})(p_{ij} - q_{ij}).$$

- as before, points are pulled towards each other if the p's are bigger than the q's, and repelled if the q's are bigger than the p's
- although symmetric SNE is slightly easier to implement, it loses the nice property of regular SNE that the data is its own optimal embedding, if the embedding dimension l is set equal to the ambient dimension d
- nevertheless, the methods seem to give similar results in practice on real datasets where $l \ll d$

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- a fundamental problem with SNE and many other embedding techniques is that they tend to squeeze points that are relatively far away in the high-dimensional space close together in the low-dimensional (usually 2D) embedding space
- this is called the *crowding problem*, and arises due to the use of squared errors (or Gaussian probabilities)

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- one solution to this is to use a probability distribution in latent space that has heavier tails, which eliminates the unwanted attractive forces between points that are relatively far in the high-dimensional space
- an obvious choice is the *Student-t distribution*:

$$\mathcal{T}(y|\mu,\sigma^2,\nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\Gamma\left(\frac{\nu}{2}\right)}\left[1+\frac{1}{\nu}\left(\frac{y-\mu}{\sigma}\right)^2\right]^{-\frac{\nu+1}{2}},$$

where:

- μ is the mean
- $\sigma > 0$ is the scale parameter (not the standard deviation)
- $\nu > 0$ is the number of *degrees of freedom* (although a better term would be the *degree of normality*, since large values of ν make the distribution act like a Gaussian)
- Γ is the *Gamma function*:

$$\Gamma(x) = \int_0^\infty t^{x-1}e^{-t}dt.$$

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- in t-SNE, the degree of freedom parameter is set to $\nu = 1$, so the distribution becomes equivalent to a Cauchy distribution:

$$q_{ij} = \frac{\left(1 + ||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}||_2^2\right)^{-1}}{\sum\limits_{\substack{k,k'=1 \\ k<k'}}^{n} \left(1 + ||\boldsymbol{z}^{(k)} - \boldsymbol{z}^{(k')}||_2^2\right)^{-1}}.$$

- we can use the same global Kullback-Leibler divergence objective as in (3)
- for t-SNE, the gradient turns out to be:

$$\nabla_{\boldsymbol{z}^{(i)}} \mathcal{L}(\boldsymbol{Z}) = 4 \sum_{j=1}^{n} (p_{ij} - q_{ij})(\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}) \left(1 + ||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}||_2^2\right)^{-1}. \tag{4}$$

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- the gradient for symmetric (Gaussian) SNE is the same, but lacks the $\left(1 + ||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}||_2^2\right)^{-1}$ term
- this term is useful because $\left(1 + ||\boldsymbol{z}^{(i)} - \boldsymbol{z}^{(j)}||_2^2\right)^{-1}$ acts like an inverse square law
- this means that points in embedding space act like stars and galaxies, forming many well-separated clusters (galaxies), each of which has many stars tightly packed inside
- this can be useful for separating different classes of data in an unsupervised way
- Figure 8 shows the application of the t-SNE algorithm to the Swiss roll dataset

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

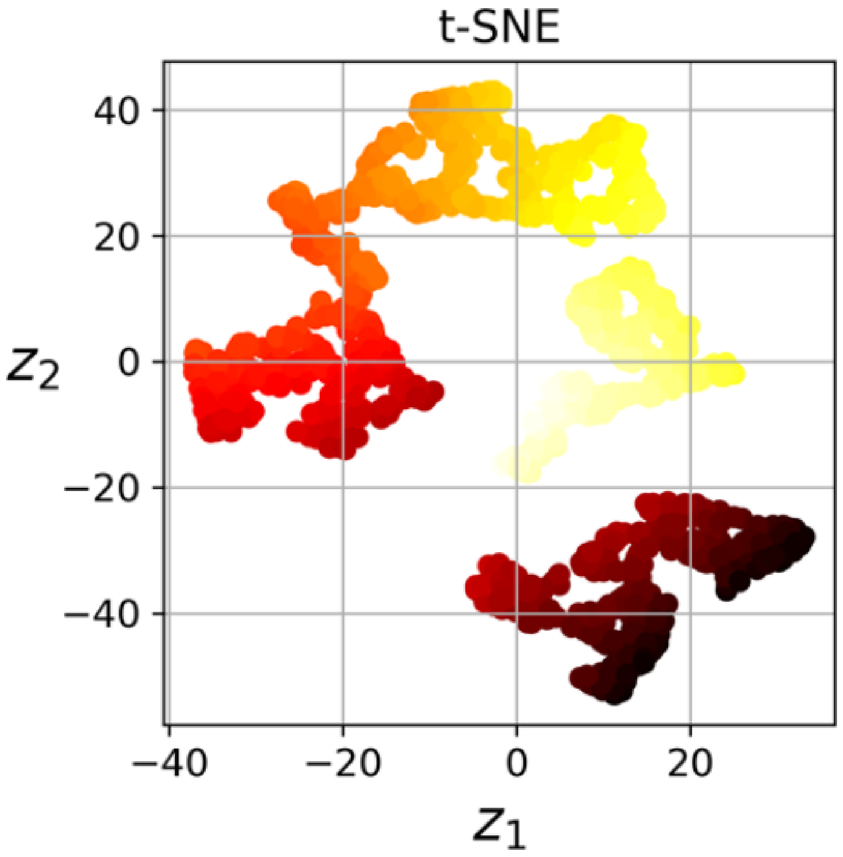


Figure 8: Using t-SNE to reduce the Swill roll to 2D.

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- an important parameter in t-SNE is the local bandwidth σ_i^2
- this is usually chosen so that P_i has a perplexity chosen by the user
- this can be interpreted as a smooth measure of the effective number of neighbors
- the *perplexity* is defined to be $2^{H(P_i)}$, where $H(P_i) = -\sum_{j=1}^{n} p_{j|i} \log_2 p_{j|i}$ is the entropy
- a big radius around each point (large value of σ_i) will result in a high entropy, and thus high perplexity

4.4 t-Distributed stochastic neighbor embedding (t-SNE)

- unfortunately, the results of t-SNE can be quite sensitive to the perplexity parameter, so it is wise to run the algorithm with many different values
- the naive implementation of t-SNE takes $\mathcal{O}(n^2)$ time, as can be seen from the gradient term in (4)
- a faster version can be created, which takes $\mathcal{O}(n \log n)$
- unfortunately, this only works well for low-dimensional embeddings, such as when $l = 2$

Exercises

Exercise 4

Assume we have the design matrix

$$\boldsymbol{X} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}.$$

Use PCA to reduce the dimension from 2 to 1, i.e., to compute the first principal component.

Exercises

Solution:

- the *singular value decomposition* (*SVD*) of a matrix $\boldsymbol{A} \in \mathbb{R}^{n \times d}$ is

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top,$$

where:

  - the columns of $\boldsymbol{U} = [\boldsymbol{u}_1 | \ldots | \boldsymbol{u}_n] \in \mathbb{R}^{n \times n}$, the left singular vectors, are the orthonormal eigenvectors of $\boldsymbol{A}\boldsymbol{A}^\top \in \mathbb{R}^{n \times n}$
  - the columns of $\boldsymbol{V} = [\boldsymbol{v}_1 | \ldots | \boldsymbol{v}_d] \in \mathbb{R}^{d \times d}$, the right singular vectors, are the orthonormal eigenvectors of $\boldsymbol{A}^\top \boldsymbol{A} \in \mathbb{R}^{d \times d}$
  - the diagonal elements of $\boldsymbol{S} = \text{diag}(s_1, \ldots, s_d)$, the singular values of $\boldsymbol{A}$, are such that $s_1^2 \geq \cdots \geq s_d^2$ are the eigenvalues of $\boldsymbol{A}^\top \boldsymbol{A}$

- by taking $\boldsymbol{A} := \frac{1}{\sqrt{n}} \boldsymbol{X}$, we have that:

  - the first l columns of $\boldsymbol{V}$ from the SVD of $\boldsymbol{X}$ are the eigenvectors $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_l$ of $\boldsymbol{\Sigma} = \frac{1}{n} \boldsymbol{X}^\top \boldsymbol{X}$
  - the corresponding eigenvalues $\lambda_1 \geq \ldots \geq \lambda_l$ are $\frac{s_1^2}{n} \geq \cdots \geq \frac{s_l^2}{n}$, where $s_1, \ldots, s_l$ are the first l singular values of $\boldsymbol{X}$

- thus, $\boldsymbol{w}_k = \boldsymbol{v}_k$ and $\lambda_k = \frac{s_k^2}{n}$, $k = 1, \ldots, l$

Exercises

- we have $n = 3$, $d = 2$ and take $\boldsymbol{A} = \frac{1}{\sqrt{3}}\boldsymbol{X} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 \\ 0 & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix}$
- now, we have that

$$\boldsymbol{A}^\top \boldsymbol{A} = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & -\frac{1}{\sqrt{3}} \\ 0 & \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 \\ 0 & \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{3}} \end{bmatrix} = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} \end{bmatrix}$$

which has eigenvalue $s_1^2 = 1$ with eigenvector $\boldsymbol{v}_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$ and eigenvalue $s_2^2 = \frac{1}{3}$ with eigenvector $\boldsymbol{v}_2 = \begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$

Exercises

- thus, $\boldsymbol{w}_1 = \boldsymbol{v}_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$, $\boldsymbol{w}_2 = \boldsymbol{v}_2 = \begin{bmatrix} -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$, $\lambda_1 = \frac{1}{3}$, $\lambda_2 = \frac{1}{9}$
- the eigenvector $\boldsymbol{w}_1$ is the *first principal component*, and has the corresponding eigenvalue $\lambda_1 = \frac{1}{3}$

Vă mulţumesc!